CAMPUS SIGNAL ISSUE TRACKER USING SINGLY LINKED LIST

Problem Statement and Project Documentation

1. Objective

The objective of this project is to develop a menu-driven C++ application to manage mobile network connectivity issues reported at different locations on a college campus. The system allows users to add, display, search, update, and delete network issue records. It uses a Singly Linked List to manage records dynamically and file handling to save records permanently.

2. Data Structure Used

Singly Linked List is a linear data structure made up of nodes. Each node stores data and a pointer to the next node.

Each node in this project stores:

• Issue ID

• Campus location

• Network provider

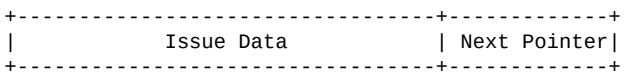
• Type of network problem

• Problem description

• Current status

• Pointer to the next node

Node representation



Why use a Singly Linked List? It supports dynamic memory allocation, allows records to be inserted and deleted, supports traversal and searching, and does not require a fixed-size array.

3. Algorithm

1. Start the program.

2. Initialize the head pointer of the linked list.

3. Load previously saved issue records from the file.

4. Display the menu and accept the user's choice.

5. Perform the selected operation: Add, View, Search, Update Status, Delete, or Exit.

6. Save updated records to the file when required.

7. Repeat the menu until the user selects Exit.

8. Release allocated memory and stop the program.

4. Working of the Code

Creating a node

The program defines a structure named Issue. Each node contains issue details and a next pointer.

Adding an issue

The program accepts the issue details, dynamically allocates memory using new, and inserts the node into the linked list.

Displaying issues

The program starts from the head pointer and follows each next pointer until the end of the list.

Searching

The user enters an issue ID. The program traverses the list and compares the ID with each node's ID.

Updating status

The program finds the specified issue and changes its status, for example from Pending to Resolved.

Deleting an issue

The program locates the record, adjusts the relevant pointers, and releases the node's memory using delete.

File handling

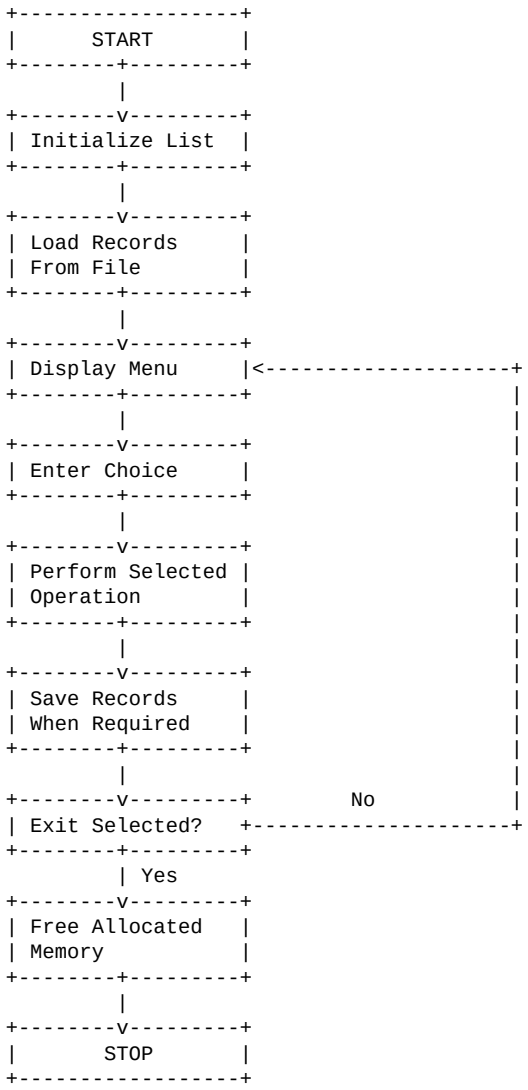
The program loads existing records from issues.txt and saves updated records so they can be retrieved in later runs.

Exiting

The program releases the memory occupied by the linked list and terminates.

5. Simple Flowchart

The following simplified flowchart represents the main program logic.



Note: This is a simplified flowchart. The actual menu handles each operation according to the user's choice.

6. Sample Output

```
===== CAMPUS SIGNAL ISSUE TRACKER =====

1. Add Issue
2. View All Issues
3. Search Issue
4. Update Issue Status
5. Delete Issue
6. Exit

Enter your choice: 1

Enter Issue ID: 101
Enter Location: College Library
Enter Network Provider: Jio
Enter Problem Type: Weak Signal
Enter Description: Poor network connectivity
Enter Status: Pending

Issue added successfully!

----- ISSUE RECORDS -----

Issue ID: 101
Location: College Library
Provider: Jio
Problem: Weak Signal
Description: Poor network connectivity
Status: Pending
```

Note: The sample output is illustrative. Match the prompts and menu text with the actual main.cpp program before final submission.

7. Conclusion

The Campus Signal Issue Tracker demonstrates the practical implementation of a Singly Linked List in C++. It provides basic operations for managing network issue records, including insertion, traversal, searching, updating, and deletion. Dynamic memory allocation makes record management flexible, while file handling allows records to be saved and retrieved across program executions. The project strengthens understanding of linked lists, pointers, functions, dynamic memory allocation, and file handling. Future improvements could include user authentication, automatic network measurements, and a graphical user interface. The current application manages manually entered reports; it does not automatically measure signal strength or send complaints to telecom operators.